

AI 서비스를 위한 SW 기초 Full-stack Engineering - 스마트 데이터 이해 및 활용

16. 종합실습 - 2

- 종합실습 1
+ CampusHub — 복합 쿼리 실습 (JOIN + 서브쿼리 + 윈도우함수)

종합 실습 - 방향

학습 목표

- 스키마 생성
→ 샘플데이터
→ **JOIN 실습**
(Inner/Left/Right/Full/Workaround)
→ 확장(Self/Cross) → 실무 집계
→ **ON vs WHERE 차이**
- 심화실습

사전 준비

- PostgreSQL 스크립트 실행
`postgres_join_lab_large`

종합실습 1 환경 활용

신규 DB 개설

**INNER/OUTER/FULL
/SELF/CROSS/ANTI
/SEMI JOIN과 집계 실습**

**심화실습
(GROUP BY, Window
function, CTE)**

종합 실습 - 배경 및 실습 과제

배경

▪ 학사 시스템: **student, enroll**

- 규칙상 수강은 학생당 0~2건
- $student_id \% 3 = 0 \rightarrow 0$ 건 (약 333명 수강 없음)
- $= 1 \rightarrow 1$ 건 (약 334명)
- $= 2 \rightarrow 2$ 건 (약 333명)
- 또한 고아 수강(enroll에만 존재)
2건(학생 1001, 1010) 포함

▪ 캠퍼스 스토어: **customers, orders**

- 고객 1명당 정확히 6건의 주문이 존재

▪ 조직도: **emp**

- CEO 1명, 매니저 10명, 직원 300명
(각 직원은 매니저 10명 중 1명에게 배정)

【 데이터 확인 및 실습과제 】

• Postgres/MySQL: LIMIT 5 / SQL Server: TOP(5)

1. 학생과 수강을 **INNER JOIN**하여
수강 존재 학생의 과목/성적을 조회
2. 모든 학생 기준으로 수강을 붙이고,
과목(없으면 **NULL**)까지 보이기
3. 수강이 기준. 학생이 없으면 학생 정보가 **NULL**
4. 학생/수강 모두 포함
5. 한 번도 수강하지 않은 학생 목록
6. 한 과목 이상 수강한 학생 목록(중복 제거)
7. 고객별 주문건수/총액
8. 총액 상위 10명과 금액
9. 모든 직원과 그 매니저 이름
10. “모든 학생 기준”으로 과목 분포를 보고 싶다
→ **LEFT JOIN** + 집계

종합 실습 - 배경 및 실습 과제(심화)

배경

- **학사 시스템: student, enroll**
 - 규칙상 수강은 학생당 0~2건
 - $\text{student_id} \% 3 = 0 \rightarrow 0\text{건}$ (약 333명 수강 없음)
 - $= 1 \rightarrow 1\text{건}$ (약 334명)
 - $= 2 \rightarrow 2\text{건}$ (약 333명)
 - 또한 고아 수강(enroll에만 존재)
2건(학생 1001, 1010) 포함
- **캠퍼스 스토어: customers, orders**
 - 고객 1명당 정확히 6건의 주문이 존재
- **조직도: emp**
 - CEO 1명, 매니저 10명, 직원 300명
(각 직원은 매니저 10명 중 1명에게 배정)

【 데이터 확인 및 실습과제(심화) 】

- **Postgres/MySQL: LIMIT 5 / SQL Server: TOP(5)**

- 11. DB 과목을 듣지 않은 모든 학생을 나열**
- 12. (가정) 과목별로 매니저가 운영 책임을 갖는다고 가정하고, emp의 매니저(이름 'Mgr_'로 시작)와 과목을 임의로 매핑한 테이블 `course_owner(course, manager_id)`를 만든 뒤, 과목별 수강 인원 + 책임 매니저 이름 리포트를 작성하세요**
- 13. 학생 × 과목 전체 조합을 만들어 "학생별 과목 추천 후보"를 만들되, 샘플 100건만 본다**

종합 실습 - 배경 및 실습 과제(심화)

배경

- **학사 시스템: student, enroll**
 - 규칙상 수강은 학생당 0~2건
 - $\text{student_id} \% 3 = 0 \rightarrow 0\text{건}$ (약 333명 수강 없음)
 - $= 1 \rightarrow 1\text{건}$ (약 334명)
 - $= 2 \rightarrow 2\text{건}$ (약 333명)
 - 또한 고아 수강(enroll에만 존재)
2건(학생 1001, 1010) 포함
- **캠퍼스 스토어: customers, orders**
 - 고객 1명당 정확히 6건의 주문이 존재
- **조직도: emp**
 - CEO 1명, 매니저 10명, 직원 300명
(각 직원은 매니저 10명 중 1명에게 배정)

【 데이터 확인 및 실습과제(심화) 】

- **Postgres/MySQL: LIMIT 5 / SQL Server: TOP(5)**

- 14. 스칼라 서브쿼리 (SELECT 절) 사용**
학생 + 소속 학과명 붙이기
- 15. 평균 GPA 보다 높은 학생 (WHERE 서브쿼리)**
- 16. 자신의 학과 평균 GPA보다 높은 학생**
(Correlated subquery)
- 17. 수강(enroll) 기록이 있는 학생만**
- 18. 한 번도 수강하지 않은 학생**
- 19. HR 학과 학생 일부와의 비교 데모**
- 20. CS 학과 학생 또는 DB 과목을 수강한 학생 목록**

종합 실습 - 배경 및 실습 과제(심화)

배경

- **학사 시스템: student, enroll**
 - 규칙상 수강은 학생당 0~2건
 - $\text{student_id} \% 3 = 0 \rightarrow 0\text{건}$ (약 333명 수강 없음)
 - $= 1 \rightarrow 1\text{건}$ (약 334명)
 - $= 2 \rightarrow 2\text{건}$ (약 333명)
 - 또한 고아 수강(enroll에만 존재)
2건(학생 1001, 1010) 포함
- **캠퍼스 스토어: customers, orders**
 - 고객 1명당 정확히 6건의 주문이 존재
- **조직도: emp**
 - CEO 1명, 매니저 10명, 직원 300명
(각 직원은 매니저 10명 중 1명에게 배정)

【 데이터 확인 및 실습과제(심화) 】

- **Postgres/MySQL: LIMIT 5 / SQL Server: TOP(5)**

- 21. student 테이블에는 학과(major)와 GPA가 있습니다.**
학과별·GPA 구간별 인원을 집계하되, 소계(소계 행)와 총계 행까지 한 번의 쿼리로 아래의 내용이 반영되도록 출력
- **GPA를 구간 (3.0 미만 / 3.0~3.5 / 3.5 초과)으로**
분류하는 파생 컬럼을 추가
 - **GROUP BY ROLLUP(major, gpa_tier)로**
학과별, 전체 소계를 동시에 조회
 - **GROUPING(major) 함수로 소계 행에**
'전체' 라벨을 붙일 것
 - **결과를 major, gpa_tier 순으로 정렬하되**
소계 행은 하단에 표시

종합 실습 - 배경 및 실습 과제(심화)

배경

- 학사 시스템: **student, enroll**
 - 규칙상 수강은 학생당 0~2건
 - $student_id \% 3 = 0 \rightarrow 0$ 건 (약 333명 수강 없음)
 - $= 1 \rightarrow 1$ 건 (약 334명)
 - $= 2 \rightarrow 2$ 건 (약 333명)
 - 또한 고아 수강(enroll에만 존재)
2건(학생 1001, 1010) 포함
- 캠퍼스 스토어: **customers, orders**
 - 고객 1명당 정확히 6건의 주문이 존재
- 조직도: **emp**
 - CEO 1명, 매니저 10명, 직원 300명
(각 직원은 매니저 10명 중 1명에게 배정)

【 데이터 확인 및 실습과제(심화) 】

- Postgres/MySQL: LIMIT 5 / SQL Server: TOP(5)

22. emp 테이블은 CEO(manager_id = NULL) → 매니저(10명) → 개발자(300명)의 3단계 계층 구조입니다.

WITH RECURSIVE로 모든 직원의 계층 경로와 깊이를 출력

- **WITH RECURSIVE**로 CEO에서 시작하는 조직 트리를 탐색

- 각 행에 **depth(0=CEO), path** 컬럼을 포함
(예: 'CEO > Mgr_2 > Dev_15')

- 매니저별 직속 부하 직원 수를 집계하는 쿼리
별도로 작성 (컬럼명 : **direct_reports**)

종합 실습 - 배경 및 실습 과제(심화)

배경

- **학사 시스템: student, enroll**
 - 규칙상 수강은 학생당 0~2건
 - $\text{student_id} \% 3 = 0 \rightarrow 0\text{건}$ (약 333명 수강 없음)
 - $= 1 \rightarrow 1\text{건}$ (약 334명)
 - $= 2 \rightarrow 2\text{건}$ (약 333명)
 - 또한 고아 수강(enroll에만 존재)
2건(학생 1001, 1010) 포함
- **캠퍼스 스토어: customers, orders**
 - 고객 1명당 정확히 6건의 주문이 존재
- **조직도: emp**
 - CEO 1명, 매니저 10명, 직원 300명
(각 직원은 매니저 10명 중 1명에게 배정)

【 데이터 확인 및 실습과제(심화) 】

- **Postgres/MySQL: LIMIT 5 / SQL Server: TOP(5)**

23. 각 학과(major)별로 GPA가 높은 순서로 순위를 매기고, 상위 3명씩만 추출하는 쿼리를 Window Function으로 작성 (서브쿼리 방식과 CTE 방식 모두 작성)

- 학과 내 순위를 계산
ROW_NUMBER() OVER (PARTITION BY major
ORDER BY gpa DESC)
- GPA가 동일한 경우 student_id 오름차순을
2차 기준으로 사용
- RANK()와 DENSE_RANK()를 함께 계산해
동점 처리 방식 차이를 비교
- 결과에 전체 학과별 학생
수(total_in_major)도 COUNT()
OVER(PARTITION BY major)로 추가

종합 실습 - 배경 및 실습 과제(심화)

배경

- **학사 시스템: student, enroll**
 - 규칙상 수강은 학생당 0~2건
 - $\text{student_id} \% 3 = 0 \rightarrow 0\text{건}$ (약 333명 수강 없음)
 - $= 1 \rightarrow 1\text{건}$ (약 334명)
 - $= 2 \rightarrow 2\text{건}$ (약 333명)
 - 또한 고아 수강(enroll에만 존재)
2건(학생 1001, 1010) 포함
- **캠퍼스 스토어: customers, orders**
 - 고객 1명당 정확히 6건의 주문이 존재
- **조직도: emp**
 - CEO 1명, 매니저 10명, 직원 300명
(각 직원은 매니저 10명 중 1명에게 배정)

【 데이터 확인 및 실습과제(심화) 】

- **Postgres/MySQL: LIMIT 5 / SQL Server: TOP(5)**

- 24. enroll 테이블을 student_id 기준으로 정렬하고, 각 학생의 이전 수강 과목 대비 성적 변화를 LAG()로 계산 (성적은 A=4, B=3, C=2, D=1 점수로 변환)**
- grade를 숫자 점수(A=4, B=3, C=2, D=1)로 변환하는 CASE 식을 작성
 - **LAG(score) OVER (PARTITION BY student_id ORDER BY course)**로 이전 과목 점수를 가져올 것
 - 현재 점수 - 이전 점수를 diff 컬럼으로 추가하고, 상승/유지/하락을 텍스트로 표시
 - 학생별 최고점과 최저점의 차이(score_range)를 Window Function으로 계산

종합 실습 - 배경 및 실습 과제(심화)

배경

- **학사 시스템: student, enroll**
 - 규칙상 수강은 학생당 0~2건
 - $\text{student_id} \% 3 = 0 \rightarrow 0\text{건}$ (약 333명 수강 없음)
 - $= 1 \rightarrow 1\text{건}$ (약 334명)
 - $= 2 \rightarrow 2\text{건}$ (약 333명)
 - 또한 고아 수강(enroll에만 존재)
2건(학생 1001, 1010) 포함
- **캠퍼스 스토어: customers, orders**
 - 고객 1명당 정확히 6건의 주문이 존재
- **조직도: emp**
 - CEO 1명, 매니저 10명, 직원 300명
(각 직원은 매니저 10명 중 1명에게 배정)

【 데이터 확인 및 실습과제(심화) 】

- **Postgres/MySQL: LIMIT 5 / SQL Server: TOP(5)**

25. orders 테이블의 주문을 order_id 순으로 정렬하여 누적 주문금액과 3개 주문 이동평균 계산 (ROWS BETWEEN 사용)

- $\text{SUM}(\text{amount}) \text{ OVER } (\text{ORDER BY order_id} \text{ ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW})$ 로 누적합 계산
- $\text{AVG}(\text{amount}) \text{ OVER } (\text{ORDER BY order_id} \text{ ROWS BETWEEN 2 PRECEDING AND CURRENT ROW})$ 로 3개 이동평균 계산
- customer_id별로 PARTITION을 나눠 고객 별 누적 구매금액도 함께 계산
- 누적합이 전체 합의 50%를 초음하는 첫 번째 order_id를 찾는 쿼리를 작성

종합 실습 - 제출물

실습 제출물
(당일 제출 원칙)

SQL Query 및 결과

- 각 문항별 쿼리와 실행결과 화면
(쿼리와 실행결과가 나온 화면 Screenshot)

제출 방법 및 배점

- Slack 내 댓글 제출 (제출기한 - 3일차 과정 시작 전)
- 문항 당 4점
- 채점 기준
 - 요구사항 반영 여부 + Query 내 컬럼 선택 (불필요한 컬럼 선택 확인)